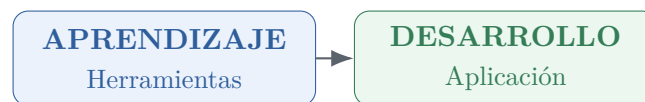


LABORATORIO 02

PARTE I

Componentes, Arduino y adquisición de datos

MATLAB/Simulink y sistema de adquisición para un motor DC



Asignatura:	Control 1
Carrera:	Ingeniería Mecatrónica
Docente:	Ing. Juan Carlos Velásquez D.
Semestre:	Semestre II – 2026
Duración:	3 Semanas

1 Propósito

En esta primera parte del Laboratorio 2 se estudiará el sistema físico utilizado para el control de un motor DC y se desarrollará la comunicación entre Arduino, MATLAB y Simulink.

El objetivo es comprender la función de cada componente y construir un sistema capaz de generar una señal de control y adquirir información mediante el encoder.

2 Objetivos

2.1 Objetivo general

Identificar experimentalmente la dinámica de un motor DC con encoder mediante MATLAB, Simulink y Arduino, obteniendo modelos en tiempo continuo y discreto para posteriormente diseñar e implementar un controlador PID sobre el sistema físico.

2.2 Objetivos específicos

1. Identificar la función y características de los componentes que conforman el sistema de control.
2. Comprender la función del Arduino dentro del sistema de adquisición y control.
3. Comprender el funcionamiento del puente H como etapa de potencia para el motor DC.
4. Comprender el funcionamiento de un encoder y su utilización para medir velocidad.
5. Utilizar el paquete de soporte de Arduino para MATLAB y Simulink.
6. Implementar la generación de PWM y la lectura del encoder mediante Simulink.
7. Obtener experimentalmente datos de entrada y salida del motor DC.
8. Identificar un modelo matemático del motor DC en tiempo continuo.
9. Identificar un modelo matemático del motor DC en tiempo discreto.
10. Comparar y validar los modelos continuo y discreto utilizando datos experimentales.
11. Diseñar un controlador PID a partir del modelo identificado.
12. Analizar el comportamiento del controlador en simulación.
13. Implementar el controlador PID de forma discreta sobre el sistema físico.
14. Comparar el comportamiento obtenido en simulación con el comportamiento experimental.



APRENDIZAJE

Construcción guiada de herramientas

Propósito de la etapa

El docente desarrollará las herramientas necesarias para interactuar con el sistema físico mediante MATLAB, Simulink y Arduino.



Docente

Se realizará una implementación guiada de cada elemento, verificando el funcionamiento individual antes de integrar todo el sistema.

3 Arquitectura del sistema

El sistema estará compuesto por:

- Arduino.
- Puente H.
- Motor DC.
- Encoder efecto Hall.
- Led.
- Potenciómetro.
- Pulsador
- Fuente de alimentación.
- Computador con MATLAB y Simulink.

4 Arduino

Arduino es una plataforma de prototipado electrónico basada en placas con microcontroladores programables. Permite leer sensores, procesar información y generar señales para controlar actuadores.

En este laboratorio, Arduino se utilizará como interfaz entre MATLAB/Simulink y el sistema físico, permitiendo generar la señal PWM para controlar el motor DC y adquirir las señales provenientes del encoder.

Enlaces de ayuda

Getting Started with Arduino

Beginner Arduino : 15 Steps (with Pictures)

Matlab Arduino

5 Puente H

El puente H sirve para controlar la dirección de giro y la velocidad (vía PWM) de motores DC mediante la alimentación de potencia y una tierra común compartida con el microcontrolador. Se usa principalmente en robótica, vehículos eléctricos, impresoras 3D y sistemas automatizados para gestionar el movimiento. Sus limitaciones son el límite de corriente máxima admitida, la pérdida de eficiencia si los transistores no entran en saturación (generando calor) y el riesgo de cortocircuitos por errores de conmutación.

Enlaces de Ayuda

H Bridge Motor Driver: Complete Engineering Guide to Topologies, ICs, and PCB Layout (2026)

H-Bridges – the Basics

Arduino genera las señales de control, pero el motor debe recibir su alimentación desde una etapa de potencia adecuada.

6 Motor DC

Los motores DC son actuadores electromecánicos donde la tensión aplicada regula directamente la velocidad de giro y dicta el cambio de su posición angular, mientras que la corriente consumida es proporcional al torque requerido para vencer la fricción mecánica y mover la carga. Sin embargo, su operación presenta limitaciones reales como la zona muerta, que es el umbral mínimo de voltaje necesario para salir del reposo, y la saturación, que limita la fuerza y velocidad máximas cuando el circuito magnético o la fuente de potencia alcanzan su capacidad límite.

7 Encoder por cuadratura

Los encoders incrementales son sensores de posición que generan pulsos eléctricos mediante dos canales A y B desfasados entre sí, lo que permite determinar el sentido de giro por cuadratura e incrementar o decrementar el conteo de pulsos. La resolución efectiva del sistema depende de la cantidad nominal de pulsos por revolución del sensor (N_{PPR}) y del modo de detección de flancos empleado por el hardware o firmware (M). Según la técnica de lectura utilizada, se pueden registrar únicamente los flancos ascendentes de un canal (modo 1x, $M = 1$), ambos flancos de un solo canal (modo 2x, $M = 2$), o todos los flancos ascendentes y descendentes de ambos canales A y B (modo 4x o cuadratura completa, $M = 4$). Así, el número total de cuentas por revolución completa viene dado por el producto $M \cdot N_{PPR}$, el cual determina la precisión con la que se calcula la posición angular acumulada y la velocidad del motor.

Considerando el factor de multiplicación de flancos M , la posición angular en el instante k se obtiene mediante:

$$\theta[k] = \frac{2\pi N[k]}{M \cdot N_{PPR}}, \quad (1)$$

mientras que la velocidad angular estimada (en rad/s) se expresa mediante la diferencia finita entre muestras:

$$\omega[k] \approx \frac{\theta[k] - \theta[k-1]}{T_s} = \frac{2\pi (N[k] - N[k-1])}{M \cdot N_{PPR} T_s}. \quad (2)$$

Asimismo, la velocidad en revoluciones por minuto (RPM) integrando el conteo de flancos

queda definida por:

$$n[k] \approx \frac{60 (N[k] - N[k-1])}{M \cdot N_{PPR} T_s}, \quad (3)$$

donde $N[k]$ representa la cuenta acumulada de flancos en el instante de muestreo k , T_s es el periodo de muestreo en segundos y $M \in \{1, 2, 4\}$ es el multiplicador de resolución según los flancos procesados.

Enlaces de Ayuda

How are magnetic rotary encoders used in industrial automation?

Quadrature Encoder Overview

Quadrature Encoders: How They Work

8 PWM

La modulación por ancho de pulso (PWM) es una técnica de salida digital que simula una señal analógica variando la duración en que una señal cuadrada se mantiene en nivel alto (1) o bajo (0). El parámetro clave es el ciclo de trabajo (duty cycle), expresado en porcentaje, el cual define la tensión promedio entregada a la carga; por ejemplo, una frecuencia constante con un ciclo del 50 % entrega la mitad del voltaje máximo de la fuente. Esta técnica se utiliza principalmente para el control de potencia, permitiendo regular la velocidad de motores DC o el brillo de un LED de forma altamente eficiente y sin generar calor excesivo por disipación.

Enlaces de Ayuda

Pulse Width Modulation

Pulse Width Modulation (PWM): How it Works and Why it's Essential in Electronics

9 Led, Potenciometro y Pulsador

LED (Salida digital/PWM): Es un diodo emisor de luz que actúa como un componente de salida para la tarjeta de control, operando bajo un principio digital (encendido/apagado) o de salida analógica simulada mediante modulación por ancho de pulso (PWM) para variar su brillo. Requiere un voltaje directo y una corriente limitada en el sentido correcto de su polaridad para funcionar.

Potenciómetro (Entrada analógica): Es un divisor de tensión variable que proporciona una entrada analógica continua. Al girar su eje, cambia la resistencia interna y entrega un nivel de voltaje proporcional dentro de un rango determinado (por ejemplo, de 0 V a 5 V), lo que permite a un microcontrolador leer magnitudes escalables como volumen, velocidad o posición deseada.

Pulsador (Entrada digital): Es un interruptor mecánico que ofrece una entrada digital discreta de dos estados (0 o 1, abierto o cerrado). Al presionarlo, conecta o interrumpe un circuito enviando una señal lógica clara a la entrada del sistema, ideal para detectar eventos instantáneos como inicios de rutina, paradas de emergencia o navegación en menús.

10 MATLAB y Arduino

1. Conexión Arduino–MATLAB

- `a = arduino(puertoCOM, modeloBoard, 'Libraries', 'RotaryEncoder')`: Inicializa la comunicación serie con la tarjeta de desarrollo (ej. 'Mega2560') cargando la librería del encoder en el objeto de conexión `a`.
- `configurePin(a, pin, modo)`: Define el modo de operación del pin especificado (como 'DigitalInput', 'DigitalOutput' o 'PWM').

2. Entradas, Salidas Digitales y Entradas Analógicas

- `readDigitalPin(a, pin)`: Lee el estado lógico actual (1 o 0) de una entrada digital.
- `writeDigitalPin(a, pin, valor)`: Establece el estado lógico (1 o 0) en una salida digital.
- `readVoltage(a, pin)`: Mide y devuelve el voltaje (de 0 a 5 V) presente en un pin de entrada analógica.

3. Generación de PWM

- `writePWMDutyCycle(a, pin, dutyCycle)`: Aplica una señal PWM en el pin seleccionado controlando el ciclo de trabajo en un rango normalizado de 0 a 1 (donde 0.5 equivale al 50 %).

4. Control de Dirección

- `writeDigitalPin(a, pinDir1, 1)` y `writeDigitalPin(a, pinDir2, 0)`: Envían estados lógicos opuestos a los pines de dirección del puente H (IN1 e IN2) para fijar el sentido de giro del motor.

5. Lectura del Encoder

- `encoderObj = rotaryEncoder(a, pinEncA, pinEncB, ppr)`: Crea e inicializa el objeto del encoder asignando los pines con interrupción y la resolución en pulsos por revolución.
- `readCount(encoderObj)`: Lee el acumulado discreto de pulsos contados por el encoder desde la última lectura o reinicio.
- `resetCount(encoderObj)`: Reinicia a cero el contador interno de pulsos del encoder.

6. Cálculo de Velocidad

- `rpmManual = (cuentas / (ppr * 4)) * (60 / dtActual)`: Estima numéricamente la velocidad en RPM calculando la diferencia de pulsos respecto a la resolución completa en cuadratura ($\text{PPR} \times 4$) y el diferencial de tiempo real (Δt).
- `readSpeed(encoderObj)`: Función nativa del objeto encoder que estima directamente la velocidad de rotación.

7. Visualización mediante Plot

- `figure`, `subplot(m, n, p)` y `plot()`: Crean la interfaz gráfica dividida en subpantallas para representar de forma sincrónica la señal PWM y las velocidades estimadas.
- `set(hLine, 'XData', x, 'YData', y)` y `drawnow`: Actualizan los datos de las curvas y fuerzan el refresco en tiempo real del gráfico durante el bucle de prueba.

11 Simulink y Arduino

1. Conexión Arduino–Simulink

- **Target Hardware Setup / Configuration Parameters:** Ajusta la configuración del modelo de Simulink para seleccionar la tarjeta objetivo (ej. Arduino Uno/Mega), definir el puerto serie COM y la tasa de baudios.
- **Deploy to Hardware / Run on Target:** Compila el diagrama de bloques en C/C++ embebido y lo descarga en el microcontrolador para su funcionamiento autónomo.
- **External Mode (Monitor & Tune):** Permite monitorear señales y sintonizar parámetros en tiempo real mientras el modelo se ejecuta físicamente en la placa.

2. Entradas, Salidas Digitales y Entradas Analógicas

- **Analog Input:** Bloque tipo *MATLAB System* que lee la tensión en un pin analógico (ej. A1 para el potenciómetro) y entrega una señal digitalizada de 10 bits (0 a 1023).
- **Digital Input:** Bloques que leen el estado lógico (1 o 0) de entradas digitales específicas (ej. pin D5 para la lectura del botón).
- **Digital Output:** Bloques que envían un valor booleano (1 o 0) a los pines de salida física asignados (ej. encendido de LED o control de dirección).

3. Generación de PWM

- **PWM:** Bloque que genera una señal PWM en un pin habilitado (ej. D4 para el motor o D6 para el LED), aceptando un ciclo de trabajo en el rango de 0 a 255.
- **Gain / Saturation:** Bloques de procesamiento aritmético para escalar y acotar la señal de control dentro de los límites válidos del bloque PWM.

4. Control de Dirección

- **Digital Output (Puente H):** Aplican las combinaciones lógicas (1/0 o 0/1) sobre los pines de dirección del puente H (ej. IN1 e IN2 en D23 y D22) para invertir la polaridad del motor.

5. Lectura del Encoder

- **Encoder:** Bloque tipo *MATLAB System* proveniente de la librería de soporte de Arduino que lee en cuadratura los canales A y B (pines D2 y D3 con interrupción) para obtener la posición en pulsos acumulados.

6. Cálculo de Velocidad

- **Discrete Derivative:** Estima la tasa de cambio temporal de los pulsos del encoder ($\Delta\theta/\Delta t$).
- **Gain:** Aplica la constante de conversión que transforma la velocidad de pulsos/s a revoluciones por minuto (RPM) o radianes por segundo (rad/s).

7. Visualización mediante Plot/Scope

- **Scope:** Muestra visualmente y en tiempo real el comportamiento de las señales de entrada, salidas PWM y respuesta de velocidad.

Producto esperado

Al finalizar el aprendizaje, el estudiante deberá ser capaz de explicar la función de cada componente y establecer una adquisición básica de velocidad mediante Arduino y Simulink.



DESARROLLO

Aplicación autónoma al problema de ingeniería

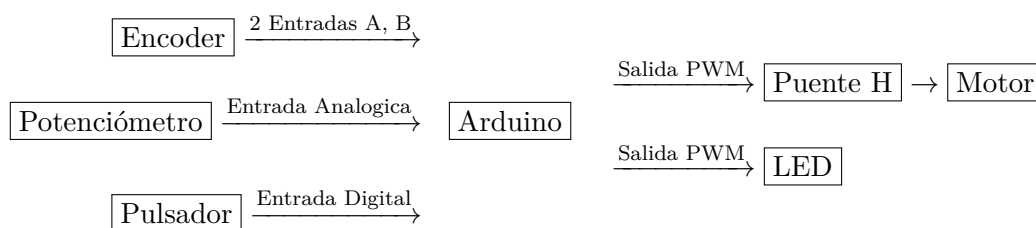
Reto de ingeniería

Construir un sistema de adquisición controlado por un pulsador (sentido de giro) y un potenciómetro (velocidad) capaz de aplicar una señal PWM al motor DC y registrar mediante el encoder la velocidad del motor.

El sistema deberá quedar preparado para utilizar los datos experimentales en la siguiente parte del laboratorio.

12 Montaje del sistema

Realizar el montaje de:



Verificar previamente las conexiones de alimentación y señales.

13 Adquisición experimental

13.1 Desarrollar un modelo de Matlab que permita:

1. Controlar la velocidad del motor por medio de un potenciómetro.
2. Generar PWM tanto para la velocidad del motor como para un LED;
3. Modificar el sentido de giro al presionar el pulsador;
4. Adquirir datos del encoder y calcular velocidad;
5. Almacenar los datos de PWM/Velocidad en MATLAB.

13.2 Desarrollar un modelo de Simulink que permita:

1. Controlar la velocidad del motor por medio de un potenciómetro.
2. Generar PWM tanto para la velocidad del motor como para un LED;
3. Modificar el sentido de giro al presionar el pulsador;
4. Adquirir datos del encoder y calcular velocidad;
5. Visualizar los datos de Referencia/PWM/Velocidad en Simulink.

14 Experimento preliminar

Aplicar diferentes valores de PWM y analizar:

- zona muerta;
- sentido de giro;
- respuesta transitoria;
- velocidad estacionaria;
- saturación.

Preguntas de desarrollo

1. ¿Qué valor mínimo de PWM permite iniciar el movimiento?
2. ¿La respuesta del motor es igual para ambos sentidos?
3. ¿Qué ocurre cuando se incrementa progresivamente el PWM?
4. ¿Qué efecto tiene el tiempo de muestreo seleccionado?



Producto esperado

El grupo deberá tener un modelo de Simulink y un script de MATLAB que realice las tareas indicadas.